

근거기반 포트폴리오

개발 온보딩 가이드

유니티(C#) 개발자를 위한 웹 풀스택 입문
— 개발 환경 구축부터 코드가 어떻게 도는지까지

이 문서는 React / Next.js / FastAPI 경험이 없는 상태를 전제로 씁니다.
유니티에서 이미 아는 개념에 하나씩 대응시켜 설명합니다.

목차

1 시작하기 전에 (이 문서를 읽는 법)	3
2 전체 그림: 무엇이 무엇과 대화하는가	3
3 개발 환경 구축 (내 PC에서 실행하기)	3
3.1 먼저 설치할 것	3
3.2 코드 받기	4
3.3 백엔드(BE) 실행	4
3.4 프론트엔드(FE) 실행	4
3.5 Redis (챗봇을 켤 때만)	5
3.6 한눈에 보는 실행 순서	5
4 백엔드 이해하기 + Swagger 사용법	5
4.1 FastAPI 와 Swagger 가 뭔가	5
4.2 API 도메인 3개	6
4.3 제공하는 API 목록	6
4.4 Swagger 로 직접 해보기 (5분 실습)	6
5 프론트엔드 코드 구조 (핵심)	7
5.1 Next.js / React 는 무엇인가	7
5.2 폴더 구조	7
5.3 가장 중요한 개념: 서버 컴포넌트 vs 클라이언트 컴포넌트	7
5.4 데이터 흐름 1: 페이지가 어떻게 채워지나	8
5.5 데이터 흐름 2: 챗봇(스트리밍)은 어떻게 도나	8
5.6 FE 는 규칙을 만들지 않는다	9
6 React 기초 — 유니티 개발자용 최소 문법	9
6.1 대응표	9
6.2 JSX: 함수가 화면을 반환한다	9
6.3 TypeScript 는 타입 있는 자바스크립트	10
7 치트시트 (자주 쓰는 명령어)	10

1 시작하기 전에 (이 문서를 읽는 법)

당신은 유니티로 게임을 만들어 왔습니다. 그 지식은 여기서도 대부분 그대로 통합니다 — “코드를 짜서 화면에 뭔가를 그리고, 데이터를 다루고, 이벤트에 반응한다” 는 본질은 같기 때문입니다. 다른 건 **도구**와 **용어**뿐입니다. 이 문서는 그 번역표를 제공합니다.

유니티에 비유하면 — 이 프로젝트는 크게 두 덩어리입니다. **프론트엔드(FE)** = 눈에 보이는 화면(유니티의 씬·UI), **백엔드(BE)** = 데이터를 주는 서버(유니티의 게임 서버·세이브 데이터). 둘은 인터넷(HTTP)으로 대화합니다.

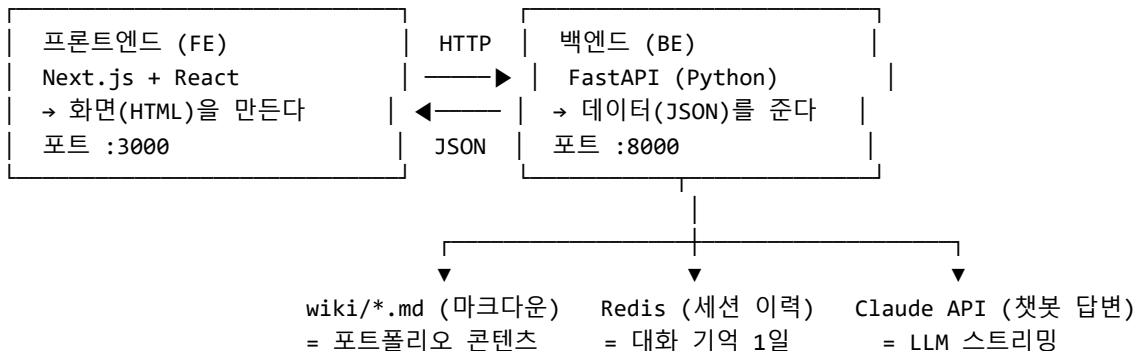
이 문서의 순서:

1. **전체 그림** — FE·BE가 어떻게 맞물리는지 (§2)
2. **환경 구축** — 내 PC에서 실제로 실행하는 법, 한 줄씩 (§3)
3. **백엔드 이해 + Swagger** — 서버 API를 문서로 보고 직접 호출해보기 (§4)
4. **프론트엔드 코드 구조** — 파일이 어떻게 짜여 있고 데이터가 어떻게 흐르는지 (§5)
5. **React 기초** — 유니티 ↔ React 대응표와 최소 문법 (§6)
6. **치트시트** — 자주 쓰는 명령어 (§7)

2 전체 그림: 무엇이 무엇과 대화하는가

[채용자의 브라우저 (Chrome 등)]

↓ HTTP 요청/응답



- **FE** 는 채용자가 보는 웹사이트를 그립니다. 스스로 데이터를 만들지 않고, 필요하면 **BE** 에 물어봐서 받아온 값으로 화면을 채웁니다.
- **BE** 는 “추천 포인트 목록 줘”, “이 프로젝트 상세 줘” 같은 요청에 **JSON 데이터**로 답합니다. 데이터의 원본은 관계형 DB 가 아니라 **마크다운 파일(wiki/)** 입니다.
- **챗봇** 질문이 오면 BE 는 published 콘텐츠를 모아 **Claude(LLM)** 에게 물어 답을 만들고, 그걸 **조금씩 흘려보내(스트리밍)** FE 에 전달합니다. 대화 기록은 **Redis** 에 잠시(1일) 저장합니다.

참고 — 왜 둘로 나눌까요? FE 는 “보기 좋게 그리기”에, BE 는 “데이터와 규칙”에 각각 집중하려는 것입니다. 유니티로 치면 **클라이언트 프로젝트**와 **전용 서버 프로젝트**를 따로 두는 것과 같습니다.

3 개발 환경 구축 (내 PC에서 실행하기)

목표: 터미널 창 **두 개**를 띄워 한쪽에서 BE(:8000), 다른 쪽에서 FE(:3000)를 돌리고, 브라우저로 `http://localhost:3000` 에 접속해 사이트를 보는 것.

3.1 먼저 설치할 것

도구	무엇	설치
----	----	----

Node.js (LTS)	FE(자바스크립트) 실행기. 유니티의 .NET 런타임에 해당	nodejs.org 또는 <code>winget install OpenJS.NodeJS.LTS</code>
Python 3.11+	BE(파이썬) 실행기	python.org 또는 <code>winget install Python.Python.3.12</code>
Redis	챗봇 세션 저장소. 챗봇을 안 쓸 거면 나중에 해도 됨	Docker 또는 Memurai (아래 §3.5)
VS Code	코드 편집기(이미 쓰는 중)	이미 설치됨

 **참고** — 설치가 됐는지 확인하려면 터미널(PowerShell)에서 `node --version`, `py --version` 을 쳐 보세요. 버전 숫자가 나오면 성공입니다.

3.2 코드 받기

이미 이 저장소를 클론했다면 생략. 아니라면:

```
git clone https://github.com/lsc892/Project_PO.git
cd Project_PO
git checkout feat/v1-implementation # 현재 구현 브랜치
```

프로젝트는 두 폴더로 나뉩니다: backend/ (BE) 와 frontend/ (FE).

3.3 백엔드(BE) 실행

파이썬은 프로젝트마다 라이브러리를 격리하는 **가상환경(venv)** 을 씁니다. 유니티에서 프로젝트별로 Packages 가 따로 있는 것과 비슷합니다.

```
cd backend
```

1) 가상환경 만들기 (한 번만)

```
py -m venv .venv
```

2) 가상환경 켜기 (터미널 열 때마다)

```
.\.venv\Scripts\Activate.ps1
```

↑ 실행 정책 오류가 나면 한 번:

```
# Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

3) 라이브러리 설치 (한 번만)

```
pip install -r requirements.txt
```

4) 환경변수 파일 만들기 – 챗봇용 Claude API 키

backend/.env 파일에 아래 한 줄 (키는 console.anthropic.com 에서 발급)

```
# ANTHROPIC_API_KEY=sk-ant-...
```

5) 서버 실행

```
uvicorn app.main:app --reload
```

성공하면 터미널에 Uvicorn running on `http://127.0.0.1:8000` 이 뜹니다. 이제 브라우저에서 `http://localhost:8000/docs` 를 열면 Swagger 문서가 보입니다(§4).

 **참고** — `--reload` 는 코드를 저장할 때마다 서버를 자동 재시작합니다. 유니티 에디터가 스크립트 변경을 감지해 다시 컴파일하는 것과 같은 편의 기능입니다.

3.4 프론트엔드(FE) 실행

새 터미널 창을 하나 더 엽니다 (BE 는 계속 돌아가야 하니까).

```
cd frontend
```

```
# 1) 라이브러리 설치 (한 번만) - node_modules 폴더가 생김  
npm install
```

```
# 2) 환경변수: BE 주소 알려주기  
# frontend/.env.local 파일에 아래 한 줄  
# NEXT_PUBLIC_API_BASE=http://localhost:8000
```

```
# 3) 개발 서버 실행  
npm run dev
```

http://localhost:3000 이 뜨면 성공. 브라우저로 접속하면 랜딩 화면이 보입니다. 콘텐츠(wiki/)가 아직 없으면 “공개된 포폴 포인트가 없습니다” 같은 빈 상태가 정상입니다.

3.5 Redis (챗봇을 켤 때만)

챗봇 기능을 실제로 테스트하려면 Redis 가 필요합니다. Windows 에서 가장 쉬운 두 방법:

```
# 방법 A) Docker (권장) - Docker Desktop 설치 후  
docker run -d -p 6379:6379 --name po-redis redis
```

```
# 방법 B) Memurai (Windows 네이티브 Redis 호환)  
winget install Memurai.MemuraiDeveloper
```

 **주의** — 화면 둘러보기(랜딩·프로젝트·포인트 상세)는 Redis 없이도 됩니다. Redis 와 ANTHROPIC_API_KEY 는 챗봇 에만 필요합니다. 없으면 챗봇만 에러가 나고 나머지는 정상입니다.

3.6 한눈에 보는 실행 순서

터미널	명령
1 (BE)	cd backend → .\venv\Scripts\Activate.ps1 → uvicorn app.main:app --reload
2 (FE)	cd frontend → npm run dev
(선택) 3	docker run -d -p 6379:6379 redis (챗봇용)
브라우저	localhost:3000 (사이트) · localhost:8000/docs (API 문서)

4 백엔드 이해하기 + Swagger 사용법

4.1 FastAPI 와 Swagger 가 뭔가

BE 는 **FastAPI** 라는 파이썬 프레임워크로 만들었습니다. FastAPI 의 큰 장점 하나: 코드를 짜면 **API 문서 (Swagger UI)**를 자동으로 만들어 줍니다. 우리가 따로 문서를 쓸 필요가 없습니다.

BE 를 켜 상태에서 브라우저로 http://localhost:8000/docs 를 열면:

- 서버가 제공하는 **모든 API 목록**이 나옵니다.
- 각 API 를 클릭하면 **어떤 값을 받고 어떤 값을 주는지** 스키마가 보입니다.
- **Try it out** 버튼으로 브라우저에서 바로 호출해 실제 응답을 볼 수 있습니다 (Postman 없이).

 **유니티에 비유하면** — Swagger UI 는 유니티의 *Inspector* + 테스트 실행기 같은 것입니다. 각 함수(API)의 입력 필드가 자동으로 그려지고, 값을 넣고 버튼을 누르면 실제로 실행돼 결과가 나옵니다.

4.2 API 도메인 3개

BE 는 세 개의 독립된 도메인으로 나뉩니다(backend/app/ 아래 폴더 = 도메인).

도메인	역할
point	포폴 포인트 = 한 개의 "결정 이야기"(STAR+ADR 9섹션 + 근거 링크). 사이트의 핵심 콘텐츠.
project	프로젝트 = 포인트들을 묶는 표지(역할·기간·기술스택 등) + 목록.
chat	챗봇 = 공개 콘텐츠를 모아 Claude 로 답을 생성, 근거와 함께 스트리밍.

각 도메인 폴더의 파일 구조는 같은 패턴입니다:

파일	책임
router.py	URL 경로 정의 (예: GET /api/points/recommended). 유니티의 이벤트 진입점.
service.py	비즈니스 규칙 (예: "추천은 featured 우선, 상위 3개"). 진짜 로직.
repository.py	데이터 읽기 (마크다운 파일을 읽어 파싱).
models.py	데이터 모양 정의 (필드와 타입). 유니티의 [Serializable] class.

4.3 제공하는 API 목록

메서드 · 경로	무엇	응답
GET /api/points/recommended	랜딩 추천 포인트 상위 3	PointSummary[]
GET /api/points?project=<slug>	한 프로젝트의 공개 포인트 목록	PointSummary[]
GET /api/points/{id}	포인트 단건(9섹션 전체)	Point · 없으면 302→/
GET /api/projects	프로젝트 카탈로그	ProjectSummary[]
GET /api/projects/{slug}	프로젝트 인덱스(표지+포인트)	ProjectIndex · 없으면 302→/
POST /api/chat	챗봇 질문 → 답변 스트리밍 (SSE)	토큰 스트림 + 근거
GET /api/chat/suggestions?point=<id>	맥락 제안 질문 3개	string[]

💡 **참고 — 302 → /** 규약: 없거나 비공개(draft) 리소스를 요청하면 서버가 "랜딩으로 가"라고 응답합니다. "그 포인트는 없다"가 아니라 그냥 홈으로 보내서 **비공개 콘텐츠의 존재 자체를 숨깁니다**.

4.4 Swagger 로 직접 해보기 (5분 실습)

1. BE 를 켜다 (uvicorn ...).
2. 브라우저 <http://localhost:8000/docs>.
3. GET /api/projects 를 펼친다 → **Try it out** → **Execute**.
4. 아래 **Response body** 에 실제 JSON 이 나온다 (콘텐츠가 없으면 []).
5. FE 페이지는 결국 이 JSON 을 받아서 화면에 그리는 것뿐 — 이 대응을 눈으로 확인하면 전체가 이해됩니다.

5 프론트엔드 코드 구조 (핵심)

5.1 Next.js / React 는 무엇인가

- **React**: 화면을 **컴포넌트**(재사용 가능한 UI 조각)로 만드는 라이브러리. 컴포넌트 = 함수 하나가 화면 일부를 반환.
- **Next.js**: React 를 실제 웹사이트로 만들어주는 상위 프레임워크 — 라우팅(URL↔파일), 서버 렌더링, 빌드 등을 담당.

🎮 **유니티에 비유하면 — 컴포넌트 = 프리팹**. 한 번 만들어 여러 곳에 인스턴스로 붙입니다.

<PointCard point={...} /> 는 "PointCard 프리팹을 이 데이터로 인스턴스화해 배치" 와 같습니다. 화면은 **컴포넌트 트리**(프리팹들의 계층) 로 구성됩니다 — 유니티 Hierarchy 와 똑같은 발상입니다.

5.2 폴더 구조

```
frontend/
├─ app/
│  ├─ layout.tsx      ← 페이지(=URL). 폴더 이름이 곧 주소가 된다.
│  ├─ page.tsx        ← 모든 페이지 공통 껍데기(헤더 + 챗봇 FAB)
│  ├─ globals.css     ← 전역 디자인(색·카드·버튼 등)
│  ├─ projects/[slug]/page.tsx ← "/projects/payments" 프로젝트 인덱스
│  │  └─ points/[id]/page.tsx   ← "/points/abc123" 포인트 상세
├─ components/        ← 재사용 UI 조각(프리팹)
│  ├─ ChatFab.tsx     ← 우하단 챗봇 열기 버튼
│  └─ chat/
│     ├─ ChatPanel.tsx ← 챗봇 패널 전체(상태·스트리밍 관리)
│     ├─ MessageList.tsx ← 말풍선 목록 + 근거 링크
│     └─ Composer.tsx  ← 입력창 + 제안 질문 칩
├─ lib/               ← 화면 아닌 순수 로직(BE 통신 등)
│  ├─ api.ts          ← BE REST 호출 래퍼(fetch)
│  ├─ chatClient.ts   ← 챗봇 SSE 스트리밍 소비
│  ├─ types.ts        ← BE 응답 데이터 타입(= BE models 의 거울)
│  └─ staticSuggestions.ts ← 무맥락 기본 제안 질문
└─ package.json       ← 의존성·스크립트(유니티의 manifest 격)
```

💡 **참고 — 파일 = URL 규칙**(Next.js App Router): app/points/[id]/page.tsx 파일이 있으면 /points/ 무엇이든 주소가 자동으로 생깁니다. [id] 의 대괄호는 "여기는 변수" 라는 뜻이고, 그 자리에 들어온 값(abc123)이 코드로 전달됩니다.

5.3 가장 중요한 개념: 서버 컴포넌트 vs 클라이언트 컴포넌트

Next.js 파일은 두 종류입니다. 이걸 구분하는 게 이 코드베이스 이해의 핵심입니다.

	서버 컴포넌트 (기본)	클라이언트 컴포넌트
표시	(아무 표시 없음)	파일 맨 위 'use client'
어디서 실행	서버에서 1번 실행 → HTML 생성	브라우저에서 실행 → 상호작용
할 수 있는 것	BE 데이터 fetch, 비동기(async)	클릭·입력·상태(useState)·타이머
이 프로젝트의 예	page.tsx (랜딩·상세)	ChatFab, ChatPanel, Composer

🎮 **유니티에 비유하면 — 서버 컴포넌트** = 게임 로딩 시 한 번 데이터를 읽어 씬을 세팅하는 것 (정적, 상호작용 없음).

클라이언트 컴포넌트 = `Update()` 가 도는 살아있는 오브젝트 (버튼 클릭·상태 변화에 반응).
버튼 클릭이나 상태 변화가 필요하면 반드시 클라이언트 컴포넌트여야 합니다.

5.4 데이터 흐름 1: 페이지가 어떻게 채워지나

랜딩 페이지 `app/page.tsx` 를 예로 보면:

```
// 서버 컴포넌트 (표시 없음) - 서버에서 실행되어 HTML을 만든다
export default async function LandingPage() {
  // 1) BE에 두 요청을 동시에 보내 데이터를 받는다
  const [recommended, projects] = await Promise.all([
    getRecommendedPoints(), // → GET /api/points/recommended
    getProjects(),          // → GET /api/projects
  ]);

  // 2) 받은 데이터로 화면(JSX)을 만든다
  return (
    <main>
      {recommended.map((p) => <PointCard key={p.id} point={p} />)}
      {projects.map((proj) => <ProjectCard key={proj.slug} project={proj} />)}
    </main>
  );
}
```

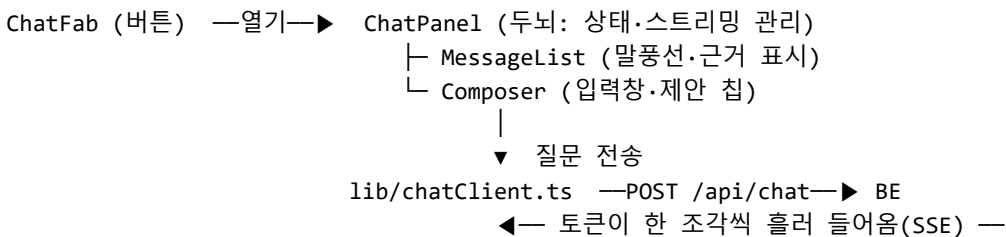
읽는 법:

- `async / await` = “느린 작업(네트워크)을 기다린다”. 유니티의 코루틴/`await Task` 와 같습니다.
- `getRecommendedPoints()` 는 `lib/api.ts` 에 있는 함수로, 실제로는 BE 에 HTTP 요청을 보냅니다.
- `.map(...)` = 배열의 각 원소를 화면 조각으로 변환. “리스트의 각 항목마다 카드 프리팹을 하나씩 생성” 과 같습니다.
- `<PointCard point={p} />` = `PointCard` 컴포넌트에 `p` 데이터를 넘겨 인스턴스화 (`point` 이 프리팹의 공개 필드).

흐름 요약: 브라우저가 / 요청 → Next.js 서버가 `LandingPage()` 실행 → BE 에서 데이터 fetch → HTML 완성해서 브라우저로 전송 → 사용자가 봄.

5.5 데이터 흐름 2: 챗봇(스트리밍)은 어떻게 도나

챗봇은 상호작용이 필요하므로 전부 **클라이언트 컴포넌트** 입니다. 관계:



`ChatPanel.tsx` 의 핵심(단순화):

`'use client';` // ← 이게 있어야 클릭·상태가 가능

```
function ChatPanel({ contextPointId, onClose }) {
  // 상태(state): 값이 바뀌면 화면이 자동으로 다시 그려진다
  const [messages, setMessages] = useState([]); // 대화 말풍선들
  const [loading, setLoading] = useState(false); // 답변 생성 중?
```



```
function send(question) {
  // 내 말풍선 + 빈 봇 말풍선을 먼저 추가
  setMessages((prev) => [...prev, {role: 'user', text: question}, {role: 'bot', text: ''}]);

  // BE로 스트리밍 요청 - 토큰이 올 때마다 봇 말풍선에 이어붙인다
  streamChat({ question, context: contextPointId }, {
    onToken: (t) => /* 마지막 봇 말풍선의 text += t */,
    onCitations: (c) => /* 답변 끝에 근거 링크 부착 */,
    onDone: () => setLoading(false),
  });
}
// ...
}
```

💡 **참고 — state(상태)의 핵심 규칙:** `useState` 로 만든 값을 `setMessages(...)` 로 바꾸면 React 가 자동으로 화면을 다시 그립니다. 유니티처럼 매 프레임 `Update()` 에서 직접 UI 를 갱신하지 않습니다 — “데이터를 바꾸면 화면은 알아서 따라온다” 가 React 의 철학입니다.

🎮 **유니티에 비유하면 — `useState` ≈ 인스펙터에 노출된 필드 + `OnValidate()`.** 값을 바꾸면 뷰가 즉시 반영됩니다.

`useEffect` ≈ `Start()` / `OnEnable()` / `OnDisable()`. “이 컴포넌트가 나타날 때/사라질 때 이걸 해라”. 이 프로젝트에선 패널이 열릴 때 입력창에 포커스, 닫힐 때 스트림 취소 등에 씁니다.

5.6 FE 는 규칙을 만들지 않는다

중요한 설계 원칙: FE 는 데이터와 규칙을 스스로 만들지 않습니다. “추천은 상위 3개”, “draft 는 숨김” 같은 판단은 전부 BE 몫이고, FE 는 받은 결과를 그리기만 합니다. `lib/types.ts` 는 BE 응답의 모양을 그대로 베낀 것이라, BE 계약이 바뀌지 않는 한 FE 도 안 바꿉니다.

6 React 기초 — 유니티 개발자용 최소 문법

6.1 대응표

유니티 / C#	React / TypeScript
프리팸	컴포넌트 (<code>function Card() { ... }</code>)
프리팸 인스턴스 배치	<code><Card ... /></code> (JSX 태그)
공개 필드 / 인스펙터 값	<code>props</code> (<code>function Card(props)</code> 로 받는 값)
인스펙터 필드 + <code>OnValidate</code>	<code>state</code> (<code>useState</code>)
<code>Start</code> / <code>OnEnable</code> / <code>OnDestroy</code>	<code>useEffect</code>
Hierarchy(오브젝트 계층)	컴포넌트 트리 (JSX 중첩)
<code>List<T></code> 순회해 UI 생성	<code>array.map(x => <Item .../>)</code>
코루틴 / <code>await Task</code>	<code>async</code> / <code>await</code>
<code>[Serializable]</code> class <code>Data</code>	<code>interface Data { ... }</code>

6.2 JSX: 함수가 화면을 반환한다

JSX 는 “자바스크립트 안에 HTML 을 섞어 쓰는 문법” 입니다. `{ }` 안에는 자바스크립트 값·표현식이 들어갑니다.

```
function PointCard({ point }) { // point = 넘겨받은 데이터(props)
  return (
    <a href={` /points/${point.id}`} > /* { } 안은 JS 값 */
      <div className="t">{point.title}</div>
      {point.tags.map((tag) => <span key={tag}>{tag}</span>)}
    </a>
  );
}
```

- className 은 HTML 의 class(CSS 스타일 이름). globals.css 의 .t, .card 등과 연결됩니다.
- {point.tags.map(...)} = 태그 배열을 각각 으로 펼침.
- key={...} = 목록 항목마다 붙이는 고유 식별자(React 가 변경을 추적하는 용도). 목록엔 항상 필요.

6.3 TypeScript 는 타입 있는 자바스크립트

C# 처럼 타입을 씁니다(string, number, Point). 덕분에 오타·잘못된 필드 접근을 편집기가 미리 잡아줍니다. 타입 검사만 돌려보려면:

```
cd frontend
npm run typecheck # = tsc --noEmit, 에러 없으면 조용히 통과
```

7 치트시트 (자주 쓰는 명령어)

하고 싶은 것	명령
BE 실행	cd backend → .\.venv\Scripts\Activate.ps1 → uvicorn app.main:app --reload
BE API 문서 보기	브라우저 http://localhost:8000/docs
FE 실행(개발)	cd frontend → npm run dev → http://localhost:3000
FE 타입 검사	npm run typecheck
FE 배포 빌드 확인	npm run build
Redis(챗봇) 켜기	docker run -d -p 6379:6379 redis
FE 라이브러리 재설치	npm install
BE 라이브러리 재설치	(venv 켜 뒤) pip install -r requirements.txt

더 깊은 계약·구조는 저장소 문서를 참고하세요:

docs/arch/ARCHITECTURE.md (기술 규약) · docs/HOME.md (도메인 지도) · docs/HANDOFF.md (구현 인계) · docs/fe/* · docs/be/* (도메인별 상세) · tickets/ (구현 티켓).